Creación de componentes visuales.

Caso práctico

Como siempre, en BK Programación están que no paran, **Juan** y **María** andan liados programado una aplicación para una empresa de venta por Internet.

-Claro, pero es que necesitamos que la interfaz disponga de un temporizador que podamos programar para implementar la cuenta atrás del tiempo que le queda a la oferta.

-Es cierto, pero, es un trabajo sencillo, no nos debe costar demasiado hacer eso...

-Cierto, además creo que hicimos algo parecido en el proyecto para el programa de juegos educativos para el instituto, necesitábamos regular el tiempo que se le daba a los alumnos para resolver el juego. Estaría muy bien recuperar ese código.

-Lo que estaría verdaderamente bien sería poder usar ese código directamente, sin necesidad de tener que buscarlo y adaptarlo...

En realidad, lo que **Juan** y **María** necesitan es adaptar el código del temporizador, que ya han programado, de manera que puedan usarlo desde la misma herramienta con la que programan sus aplicaciones, y para ello una de las mejores opciones es el uso de **componentes**.





GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

Aviso Legal

1.- Concepto de componente. Características.

Caso práctico

Ada ha escuchado la conversación de sus compañeros, e interviene para explicarles qué es un componente:

-Un componente es una pieza de hardware o software pequeña, que tiene un comportamiento específico y dispone de una interfaz que permite que se inserte fácilmente. Por ejemplo, una etiqueta o un botón que se basan en las etiquetas o botones estándares de la paleta de NetBeans adaptados a las necesidades específicas de una aplicación y que puede ser reutilizada es un componente. Tienen la ventaja de que pueden especializarse mucho y al estar probadas de antemano sabemos que su funcionamiento es correcto por lo que facilitan la programación de aplicaciones nuevas.



Un **componente software** es una clase creada para ser reutilizada y que puede ser manipulada por una herramienta de desarrollo de aplicaciones visual. Se define por su **estado** que se almacena en un conjunto de **propiedades**, las cuales pueden ser modificadas para adaptar el componente al programa en el que se inserte. También tiene un **comportamiento** que se define por los eventos ante los que responde y los **métodos** que ejecuta ante dichos eventos.

Un subconjunto de los atributos y los métodos forman la **interfaz** del componente.

Para que pueda ser distribuida se empaqueta con todo lo necesario para su correcto funcionamiento, quedando independiente de otras bibliotecas o componentes.

Citas para pensar

> Un componente es una unidad de composición de software, que posee un conjunto de interfaces y un conjunto de requisitos, y que ha de poder ser desarrollado, adquirido, incorporado al sistema y compuesto de otros componentes, de forma independiente en tiempo y en espacio.
>
> *Szypersky, 1998*

Para que una clase sea considerada un componente debe cumplir ciertas normas:

- Debe poder modificarse para adaptarse a la aplicación en la que se integra.
- Debe tener persistencia, es decir, debe poder guardar el estado de sus propiedades cuando han sido modificadas.
- Debe tener introspección, es decir, debe permitir a un IDE que pueda reconocer ciertos elementos de diseño como los nombres de las funciones miembros o métodos y definiciones de las clases, y devolver esa información.
- Debe poder gestionar **eventos**.

El desarrollo basado en componentes tiene, además, las siguientes **ventajas**:

- Es mucho más sencillo, se realiza en menos tiempo y con un coste inferior.

- Se disminuyen los errores en el software ya que los componentes se deben someter a un riguroso control de calidad antes de ser utilizados.

Reflexiona

Ya has creado algunas interfaces gráficas con diferentes herramientas como NetBeans o Designer de QT. En ambos casos, el procedimiento consiste en seleccionar los controles gráficos de la interfaz y posicionarlos en un lienzo que será la interfaz. Todos estos controles **cumplen con los requisitos para ser componentes, de hecho lo son**, son elementos reutilizables que pueden ser manejados por una herramienta de desarrollo visual. Por ejemplo, se puede modificar su tamaño o color para adaptarlo a la interfaz y estos cambios permanecen después de cerrarla, además tienen una interfaz formada por un conjunto de métodos y propiedades accesibles desde la paleta de propiedades y la capacidad de responder a eventos, por ejemplo el clic del ratón.

Autoevaluación

Imagina una herramienta de desarrollo, están compuestas de un conjunto de ventanas que podemos mostrar u ocultar, cambiar su tamaño, etc., de modo que al cerrar la aplicación y volverla a abrir se mantiene la estructura que tenía al cerrarse. ¿Con qué característica de los componentes relacionas esto?

- ○ Con la persistencia.
- ○ Con la introspección.
- ○ Con la gestión de eventos.

2.- Elementos de un componente: propiedades y atributos.

Caso práctico

Ada continúa explicando a sus compañeros las características más básicas de un componente:

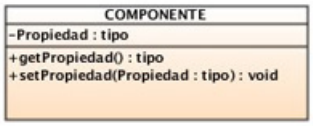
-Un componente software no deja de ser una clase que vamos a tratar de una forma un poco diferente, así que como todas las clases se compone de un conjunto de elementos de tipos básicos a los que se llama atributos, pero si los atributos cumplen ciertas características pasarán a ser propiedades. Por ejemplo, una etiqueta es un componente, las características que la definen son sus propiedades como el color o el tipo de la fuente. Además tiene un conjunto de métodos o funciones que permiten modificar las propiedades, y para implementar la relación con otros elementos software o con el usuario le podemos definir una serie de eventos.



Como en cualquier clase, un componente tendrá definido un estado a partir de un conjunto de **atributos**. Los atributos son variables definidas por su nombre y su tipo de datos que toman valores concretos. Normalmente los atributos son privados y no se ven desde fuera de la clase que implementa el componente, se usan sólo a nivel de programación.

> **Las propiedades son un tipo específico de atributos que representan características de un componente que afectan a su apariencia o a su comportamiento. Son accesibles desde fuera de la clase y forman parte de su interfaz. Suelen estar asociadas a un atributo interno.**

Las propiedades de un componente pueden examinarse y modificarse mediante métodos o funciones miembro, que acceden a dicha propiedad, y pueden ser de dos tipos:



- getter: permiten leer el valor de la propiedad. Tienen la estructura:

```
public  get( )
```

  si la propiedad es booleana el método getter se implementa así:

```
public boolean is()
```

- setter: permiten establecer el valor de la propiedad. Tiene la estructura:

```
public void set( valor)
```

Si una propiedad no incluye el método set entonces es una propiedad de **sólo lectura**.

Por ejemplo, si estamos generando un componente para crear un botón circular con sombra, podemos tener, entre otras, una propiedad de ese botón que sea **color**, que tendría asociados los siguientes métodos:

```
public void setColor(String color)
public String getColor()
```

Reflexiona

En el lenguaje de programación Java los componentes se crean utilizando la tecnología **JavaBeans**, que consiste en crear una clase con unas características especiales que puede ser reutilizada después de una manera muy sencilla. De hecho es común que a un componente Java se le llame Bean. A continuación, tienes un enlace a un ejemplo de creación de un Bean.

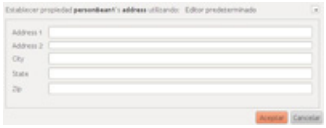
http://www.youtube.com/watch?v=jcQpkKCEyVs
http://docs.oracle.com/javase/tutorial/javabeans/

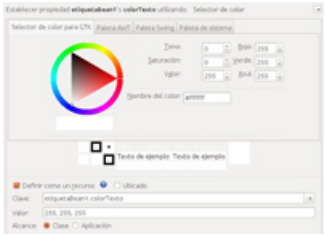
Realiza los pasos del tutorial y analiza cuál es la función principal de un componente.

2.1.- Modificar gráficamente el valor de una propiedad con un editor.

Una de las principales características de un componente es que una vez instalado en un entorno de desarrollo, éste debe ser capaz de identificar sus propiedades simplemente detectando parejas de operaciones get/set, mediante la capacidad denominada **introspección**.

El entorno de desarrollo podrá editar automáticamente cualquier propiedad de los tipos básicos o de las clases **Color** y **Font**. Aunque no podrá hacerlo si el tipo de datos de la propiedad es algo más complejo, por ejemplo, si usamos otra clase, como Cliente. Para poder hacerlo tendremos que crear nuestro propio editor de propiedades, por ejemplo, en la primera imagen puedes ver un editor programado para un componente que almacena información de personas, el nombre o el teléfono. Son cadenas de caracteres que se editan fácilmente, pero la dirección es una propiedad compuesta que precisa del siguiente editor. En la segunda imagen aparece el editor de una propiedad de tipo **Color**.





> Un **editor de propiedad** es una herramienta para personalizar un tipo de propiedad en particular. Los editores de propiedades se utilizan en la ventana Propiedades, que es donde se determina el tipo de la propiedad, se busca un editor de propiedades apropiado, y se muestra el valor actual de la propiedad de una manera adecuada a la clase.

La creación de un editor de propiedades usando tecnología Java supone programar una clase que implemente la interfaz **PropertyEditor**, que proporciona métodos para especificar cómo se debe mostrar una propiedad en la hoja de propiedades. Su nombre debe ser el nombre de la propiedad seguido de la palabra Editor:

```
public Editor implements PropertyEditor {…}
```

Por defecto la clase **PropertyEditorSupport** que implementa **PropertyEditor** proporciona los editores más comúnmente empleados, incluyendo los mencionados tipos básicos, **Color** y **Font**.

Una vez que tengamos todos los editores, tendremos que empaquetar las clases con el componente para que use el editor que hemos creado cada vez que necesite editar la propiedad. Así, conseguimos que cuando se añada un componente en un panel y lo seleccionemos, aparezca una hoja de propiedades, con la lista de las propiedades del componente y sus editores asociados para cada una de ellas. El IDE llama a los métodos **getters**, para mostrar en los editores los valores de las propiedades. Si se cambia el valor de una propiedad, se llama al método **setter**, para actualizar el valor de dicha propiedad, lo que puede o no afectar al aspecto visual del componente en el momento del diseño.

Para saber más

Si quieres conocer un poco más de este apartado de la tecnología Java puedes consultar la página del tutorial que tiene Oracle alojado en su página (está en inglés) y la especificación de la interfaz **PropertyEditor**:

Interfaz PropertyEditor.

Autoevaluación

Si necesitamos crear un editor para una propiedad que se llame `dirección`, indica cuál es la manera correcta de definir la clase para programarlo:

- `private direccion implements PropertyEditor{...}.`
- `public direccionEditor {...}.`
- `public direccionEditor implements PropertyEditor {...}.`

2.1.1.- Ejemplo de creación de un componente con un editor de propiedades.

Citas para pensar

Un **JavaBean** es un componente software reusable que puede ser manipulado visualmente mediante una herramienta gráfica.

Especificación de JavaBeans, Sun 1997

Los editores de propiedades tienen dos propósitos fundamentalmente:

- Convertir el valor de "y" a cadenas para ser mostrados adecuadamente conforme a las características de la propiedad, y
- Validar los datos nuevos cuando son introducidos por el usuario.

Los pasos básicos para crear un editor de propiedades consisten en:

1. Crear una clase que extienda a `PropertyEditorSupport`.
2. Añadir los métodos `getAsText` y `setAstext`, que transformarán el tipo de dato de la propiedad en cadena de caracteres o viceversa.
3. Añadir el resto de métodos necesarios para la clase.
4. Asociar el editor de propiedades a la propiedad en cuestión.



Para ilustrar como se implementa un componente con una herramienta como NetBeans, adaptaremos un campo de texto de manera que podamos modificar mediante propiedades el **ancho** que ocupa en el formulario (número de columnas), el **color** del texto y la **fuente**. De esta manera veremos como modificar, desde NetBeans, propiedades sencillas que no necesitan un editor. Además, vamos a añadir una propiedad, denominada **tipo**, que aunque es una cadena de caracteres requiere de un editor de propiedades porque solo puede tomar tres posibles valores que se seleccionarán mediante una lista desplegable. Esta última propiedad contiene un tipo de dato, de manera que sólo se puedan escribir valores enteros, reales o Texto en el campo de texto. Finalmente tendremos las siguientes propiedades:

- El **ancho** que se puede representar como una propiedad de tipo entero.
- El **color** que será una propiedad de tipo `Color`, que como hemos dicho cuenta con su propio editor.
- La **fuente** será de tipo `Font`, y también cuenta con su propio editor.
- El **tipo** datos será una propiedad compuesta por el tipo de datos y el contenido del campo de texto en si que necesitará que programemos un editor de propiedades para el.

Veamos como crear este ejemplo.

Debes conocer

La siguiente presentación te indicará los pasos que debes seguir para crear el campo de texto que acabamos de describir:

Resumen textual alternativo

Y aquí tienes el proyecto de NetBeans para que puedas descargarlo. (55.2 KB)

2.2.- Propiedades simples e indexadas.

Caso práctico

Los chicos están ya funcionando a pleno rendimiento con componentes, pero **Juan** ha encontrado un pequeño escollo:

-Estoy programando un componente para dibujar un menú en una interfaz, sin embargo, si quiero que el número de elementos pueda ser definido después, ¿cómo implemento la estructura para almacenar el contenido del menú, si a priori no sé cuánto elementos voy a tener?

-Puedes usar un vector como habrías hecho en cualquier otro caso, –**Ada** siempre está al quite-, después sólo tienes que cambiar los métodos getter y setter para adaptarlos a un tipo de datos compuesto.





- Una **propiedad simple** representa un único valor, un número, verdadero o falso o un texto por ejemplo.
  Tiene asociados los métodos `getter` y `setter` para establecer y rescatar ese valor. Por ejemplo, si un componente de software tiene una propiedad llamada peso de tipo real susceptible de ser leída o escrita, deberá tener los siguientes métodos de acceso:

```
public real getPeso()
public void setPeso(real nuevoPeso)
```

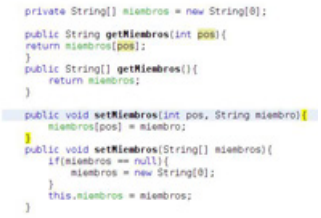
  Una propiedad simple es de sólo lectura o sólo escritura si falta uno de los mencionados métodos de acceso.

- Una **propiedad indexada** representa un conjunto de elementos, que suelen representarse mediante un vector y se identifica mediante los siguientes patrones de operaciones para leer o escribir elementos individuales del vector o el vector entero (fíjate en los corchetes del vector):




```
public [] get()
public void set ([] p)
public  get(int posicion)
public void set (int posicion,  p)
```

Aquí tienes un ejemplo de propiedad indexada que resuelve el problema de Juan. El componente tiene una propiedad indexada que almacena los contenidos del menú, de este modo es sencillo crear diferentes menús de distintos tamaños. Para acceder a el solo tienes que definir dos tipos de setter y getter.



```
private String[] miembros = new String[0];

public String getMiembros(int pos){
return miembros[pos];
}
public String[] getMiembros(){
    return miembros;
}

public void setMiembros(int pos, String miembro){
    miembros[pos] = miembro;
}
public void setMiembros(String[] miembros){
    if(miembros == null){
        miembros = new String[0];
    }
    this.miembros = miembros;
}
```

Resumen textual alternativo

A continuación puedes descargar el código asociado para que lo estudies:

Código para una propiedad indexada. (2.27 KB)

Autoevaluación

¿Que tipo de propiedad usarías para implementar el teléfono móvil de una persona?

- ○ Una propiedad indexada.
- ○ Una propiedad simple.

2.3.- Propiedades compartidas y restringidas.

Caso práctico

Juan continúa comentando sus problemas de propiedades a **Ada**.

-¿Y qué me recomiendas si quiero que me muestren diferentes páginas según el elemento seleccionado? Tengo un componente para el menú y otro para el contenido que se llama panel.

-Puedes usar una propiedad compartida entre ambos que haga que el contenido cambie cuando se modifique el menú.



Los objetos de una clase que tiene una **propiedad compartida o ligada** notifican a otros objetos oyentes interesados, cuando el valor de dicha propiedad cambia, permitiendo a estos objetos realizar alguna acción. Cuando la propiedad cambia, se crea un objeto (de una clase que hereda de `ObjetEvent`) que contiene información acerca de la propiedad (su nombre, el valor previo y el nuevo valor), y lo pasa a los otros objetos oyentes interesados en el cambio.



La notificación del cambio se realiza a través de la generación de un **PropertyChangeEvent**. Los objetos que deseen ser notificados del cambio de una propiedad limitada deberán registrarse como auditores. Así, el componente de software que esté implementando la propiedad limitada suministrará métodos de esta forma:

```
public void addPropertyChangeListener (PropertyChangeListener l)
public void removePropertyChangeListener (PropertyChangeListener l)
```

Los métodos precedentes del registro de auditores no identifican propiedades limitadas específicas. Para registrar auditores en el **PropertyChangeEvent** de una propiedad específica, se deben proporcionar los métodos siguientes:

```
public void addPropertyNameListener (PropertyChangeListener l)
public void removePropertyNameListener (PropertyChangeListener l)
```

En los métodos precedentes, **PropertyName** se sustituye por le nombre de la propiedad limitada. Los objetos que implementan la interfaz **PropertyChangeListener** deben implementar el método **propertyChange()**. Este método lo invoca el componente de software para todos sus auditores registrados, con el fin de informarles de un cambio de una propiedad.

> Una **propiedad restringida** es similar a una propiedad ligada salvo que los objetos oyentes que se les notifica el cambio del valor de la propiedad tienen la opción de vetar cualquier cambio en el valor de dicha propiedad.

Los métodos que se utilizan con propiedades simples e indexadas que veíamos anteriormente se aplican también a las propiedades restringidas. Además, se ofrecen los siguientes métodos de registro de eventos:

```
public void addPropertyVetoableListener (VetoableChangeListener l)
public void removePropertyVetoableListener (VetoableChangeListener l)
public void addPropertyNameListener (VetoableChangeListener l)
public void removePropertyNameListener (VetoableChangeListener l)
```

Los objetos que implementa la interfaz **VetoableChangeListener** deben implementar el método **vetoableChange()**. Este método lo invoca el componente de software para todos sus auditores registrados con el fin de informarles del cambio en una propiedad.

Todo objeto que no apruebe el cambio en una propiedad puede arrojar una **PropertyVetoException** dentro del método **vetoableChange()** para informar al componente cuya propiedad restringida hubiera cambiado de que el cambio no se ha aprobado.

Autoevaluación

Supongamos que Juan está implementando su aplicación de interfaz gráfica con un componente `Tmenu` que contiene las opciones del programa y un componente panel que usa para mostrar una página asociada a la opción seleccionada, el menú tiene una propiedad que se denomina OpcionActiva que cambia cada vez que el usuario hace clic sobre una opción concreta, ¿qué tipo de propiedad debe usar para implementar OpcionActiva?

- ○ Propiedad compartida.
- ○ Restringida.

3.- Eventos. Asociación de acciones a eventos.

Caso práctico

Los componentes de **Juan** van cobrando forma...

—De momento tengo bastante bien definida la parte estática del componente, sus propiedades y atributos y los métodos que me sirven para poder gestionarlos, pero en realidad mis componentes no hacen nada aún...



Juan tiene razón, la funcionalidad de un componente viene definida por las acciones que puede realizar definidas en sus métodos y no solo eso, también se puede programar un componente para que reaccione ante determinadas acciones del usuario, como un clic del ratón o la pulsación de una tecla del teclado. Cuando estas acciones se producen, se genera un **evento** que el componente puede capturar y procesar ejecutando alguna función. Pero no solo eso, un componente puede también lanzar un evento cuando sea necesario y que su tratamiento se realice en otro objeto.

Los eventos que lanza un componente, se reconocen en las herramientas de desarrollo y se pueden usar para programar la realización de acciones.

Para que el componente pueda reconocer el evento y responder ante él, tendrás que hacer lo siguiente:

- Crear una clase para los eventos que se lancen.
- Definir una interfaz que represente el oyente (`listener`) asociado al evento. Debe incluir una operación para el procesamiento del evento.
- Definir dos operaciones, para añadir y eliminar oyentes. Si queremos tener más de un oyente para el evento tendremos que almacenar internamente estos oyentes en una estructura de datos como `ArrayList` o `LinkedList`:

```
public void addListener(Listener l)
public void removeListener(Listener l)
```

- Finalmente, recorrer la estructura de datos interna llamando a la operación de procesamiento del evento de todos los oyentes registrados.

En resumen:



Resumen textual alternativo

Autoevaluación

Otra manera de implementar la gestión del cambia de la propiedad OpcionActiva del menú de Juan podría ser...

- ○ ...mediante una propiedad especial.
- ○ ...programando un método especial para ello.
- ○ ...con los eventos.

3.1.- Ejemplo de gestión de eventos.

Para ver como funciona la gestión de eventos sobre componentes gráficos vamos a completar el ejemplo que estábamos viendo del campo de texto con tipo. Si recuerdas, se trataba de un componente que hereda de un campo de texto genérico.

Nuestro componente dispone, entre otras, de una propiedad llamada texto, que sólo puede tomar los valores "Entero", "Real" y "Texto".

El objetivo de este ejemplo es comprobar que cuando se escribe un dato éste coincida con el tipo que se indica.

Para implementar éste, vamos a hacer uso de un evento predefinido para los campos de texto llamado **KeyEvent**, que se dispara cuando se pulsa una tecla sobre el control. Para gestionarlo vamos a crear una función llamada **gestionaEntrada** que convertirá a nuestro componente en escuchador del evento mediante la sentencia

```
this.addKeyListener(new KeyAdapter()
```

y programará el método **KeyTiped** que gestiona qué hacer con la tecla pulsada en función de si el tipo es Entero o Real, si es Texto no se gestionará la entrada porque se asume como válido cualquier texto.

Por último, se llama a este método desde el constructor para que se tenga en cuenta al generar un componente nuevo.



Resumen textual alternativo

Para saber más

A continuación puedes descargar el código asociado para que lo estudies:

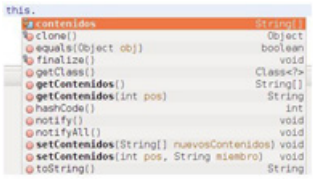
Código del componente con un evento implementado. (4.22 KB)

4.- Introspección. Reflexión.

Caso práctico

Ada quiere aclarar algunas propiedades más relacionadas con los componentes.

-Cuando trabajamos con cualquier clase, y los componentes no dejan de serlo, el editor proporciona algunas ayudas a la edición. Te dan pistas acerca de los métodos que puedes usar o qué argumentos debes colocar en una función. ¿Cómo puede la herramienta conocer eso? Sencillamente porque un componente, como cualquier otra clase dispone de una interfaz, que es el conjunto de métodos y propiedades accesibles desde el entorno de programación. Normalmente, la interfaz la forman los atributos y métodos públicos.



La **introspección** es una característica que permite a las herramientas de programación visual arrastrar y soltar un componente en la zona de diseño de una aplicación y determinar dinámicamente qué métodos de interfaz, propiedades y eventos del componente están disponibles.



Esto se puede conseguir de diferentes formas, pero en el nivel más bajo se encuentra una característica denominada reflexión, que busca aquellos métodos definidos como públicos que empiezan por get o set, es decir, se basa en el uso de patrones de diseño, o sea, en establecer reglas en la construcción de la clase de forma que mediante el uso de una nomenclatura específica se permita a la herramienta encontrar la interfaz de un componente.

También se puede hacer uso de una clase asociada de información del componente (BeanInfo) que describe explícitamente sus características para que puedan ser reconocidas.

Para saber más

En el siguiente enlace podrás acceder a una página web de Introducción a los JavaBeans donde podrás ampliar los conceptos de introspección, persistencia, reflexión, etc. de componentes visuales en Java.

Características de los JavaBeans.

Autoevaluación

La reflexión permite....

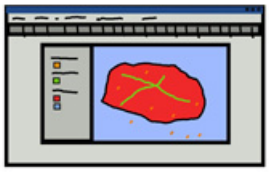
- ○ … a una herramienta de desarrollo detectar los nombres de las propiedades y métodos de la interfaz de una clase.
- ○ ...facilitar la implementación de los métodos de una clase.
- ○ ...conocer los elementos de la interfaz de un componente mediante el uso de una clase llamada `BeanInfo`.

5.- Persistencia del componente.

Caso práctico

Ada continúa hablando sobre los componentes...

—Fijaos en editor de texto que utilizáis a diario, o el IDE con el que desarrolláis vuestros programas, cada uno lo utiliza de una forma, a **María** le gusta dejar el máximo espacio para poder escribir sus programas, sin embargo Juan necesita tener cuantos más paneles y barras de herramientas mejor, y cada mañana cuando encendéis el equipo la configuración que habéis establecido continúa ahí, y podéis continuar vuestro trabajo en lugar de tener que empezar cambiar, añadir y quitar cosas, con los componentes pasa algo parecido.



A veces, necesitamos almacenar el estado de una clase para que perdure a través del tiempo. A esta característica se le llama **persistencia**. Para implementar esto, es necesario que pueda ser almacenada en un archivo y recuperado posteriormente.

El mecanismo que implementa la persistencia se llama **serialización**.

Al proceso de almacenar el estado de una clase en un archivo se le llama **serializar**.



Al de recuperarlo después **deserializar**.



Todos los componentes deben persistir. Para ello, siempre desde el punto de vista Java, deben implementar los interfaces **java.io.Serializable** o **java.io.Externalizable** que te ofrecen la posibilidad de serialización automática o de programarla según necesidad:

- **Serialización Automática**: el componente implementa la interfaz `Serializable` que proporciona serialización automática mediante la utilización de las herramientas de `Java Object Serialization`. Para poder usar la interfaz serializable debemos tener en cuenta lo siguiente:
  - Las clases que implementan `Serializable` deben tener un **constructor sin argumentos** que será llamado cuando un objeto sea "reconstituido" desde un fichero .ser.
  - Todos los campos excepto `static` y `transient` son serializados. Utilizaremos el modificador `transient` para especificar los campos que no queremos serializar, y para especificar las clases que no son serializables (por ejemplo `Image` no lo es).
  - Se puede programar una **serialización propia** si es necesario implementando los siguientes métodos (las firmas deben ser exactas):

    - `private void writeObject(java.io.ObjectOutputStream out) throws IOException;`
    - `private void readObject(java.io.ObjectInputStream in) throws IOExcept`
- **Serialización programada**: el componente implementa la interfaz **Externalizable**, y sus dos métodos para guardar el componente con un formato específico. Características:
    - Precisa de la implementación de los métodos `readExternal()` y `writeExternal()`.
    - Las clases `Externalizable` también deben tener un constructor sin argumentos.

Los componentes que implementarás en esta unidad emplearán la **serialización por defecto** por lo que debes tener en cuenta lo siguiente:

- La clase debe implementar la interfaz `Serializable`.
- Es obligatorio que exista un **constructor sin argumentos**.

Para saber más

La documentación sobre la interfaz serializable de java la puedes encontrar en el siguiente enlace:

Serialización Java.

También tienes un pequeño artículo sobre los secretos de la serialización con algunos ejemplos

Los secretos de la serialización Java.

6.- Otras tecnologías para la creación de componentes visuales.

Caso práctico

—Hemos visto la tecnología JavaBean para la generación de componentes —comenta **Juan**—, ¿qué otras podemos encontrar?



En la actualidad existen diversas tecnologías para la creación de componentes visuales, además de la que hemos visto a lo largo del tema, JavaBeans implementados con NetBeans podemos encontrar otras orientaciones como los estándares COM, COM+ y DCOM de Microsoft y CORBA del Object Management Group.

En general, se trata de diversas maneras de ofrecer los servicios de persistencia e introspección, para un modelo orientado a objetos, de modo que se puedan crear clases reutilizables de las que se conozca su interfaz, quedando oculta su implementación.

JavaBeans de Oracle

Es el estándar para crear componentes proporcionado por Oracle. Su características más destacadas las hemos visto a los largo de la unidad, son clases Java que implementan la interfaz `Serializable` y deben disponer de un constructor sin argumentos. Define como gestionar la persistencia y la introspección y soporta propiedades simples, indexadas, compartidas o restringidas así como la gestión de eventos.

Con esta tecnología la creación de componentes visuales es realmente sencilla, basta con heredar de un control visual que ya exista, como una imagen, o una etiqueta, implementando la interfaz `Serializable` al mismo tiempo.

Como toda la tecnología Java es software libre.

El Modelo de Objetos Componentes de Microsoft.

Es la tecnología propuesta por Microsoft para la creación de componentes. Forman parte de ella COM, DCOM, COM+, OLE y ActiveX. Se basa en la creación de objetos que tiene una interfaz bien definida e independiente de la implementación de forma que pueden ser reutilizados sin más que conocer su interfaz en entornos distintos a aquel en el que fue creado. Usando DCOM además pueden estar distribuidos en varias máquinas, y COM+ usa un servidor de componentes denominado MTS.

La principal ventaja de estos componentes es que al estar separado su interfaz de su implementación se pueden usar desde diferentes lenguajes de programación, de hecho Java, Microsoft Visual C++, Microsoft Visual Basic, Delphi, PowerBuilder, y Micro Focus COBOL interactúan perfectamente con DCOM.

Aunque esta tecnología se ha implementado en muchas plataformas se usa fundamentalmente en Microsoft Windows siendo distribuido con licencia propietaria.

Para saber más

En la siguiente página web encontrarás toda la información acerca de la creación de componentes de Microsoft.

Creación de componentes.

CORBA

Es un estándar definido por el Object Management Group que permite añadir una "envoltura" a un código escrito en un determinado lenguaje con información de las capacidades que el código contiene y de sus métodos de forma que puedan ser descubiertos. Se programa en un lenguaje específico, IDL, del que existen implementaciones para diferentes lenguajes orientados a objetos, como Ada, C++, Perl, Java, SmallTalk, etc.

Para saber más

En la siguiente página web encontrarás información acerca de la creación de componentes CORBA.

Creación de componentes CORBA.

Autoevaluación

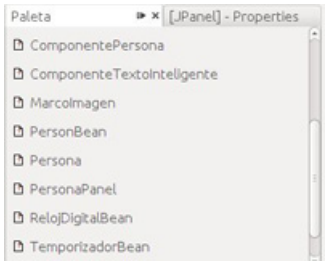
Sea cual sea la tecnología empleada para crear un componente, ¿cuál crees que es la principal característica que debe cumplir?

- ○ Que sea orientado a objetos.
- ○ Que permita la gestión de eventos.
- ○ Que permita reconocer la interfaz de un objeto con independencia de su implementación.

7.- Empaquetado de componentes.

Caso práctico

Juan está terminando el componente, ha revisado las propiedades y sus métodos, y también los gestores de eventos (su implementación y los escuchadores) y ahora necesita poder disponer del componente en la paleta para poder usarlo.



Una vez creado el componente, es necesario empaquetarlo para poder distribuirlo y utilizarlo después. Para ello necesitarás el paquete jar que contiene todas las clases que forman el componente:

- El propio componente
- Objetos `BeanInfo`
- Objetos `Customizer`
- Clases de utilidad o recursos que requiera el componente, etc.

Puedes incluir varios componentes en un mismo archivo.

El paquete **jar** debe incluir un fichero de manifiesto (con extensión **.mf**) que describa su contenido, por ejemplo:

En este documento encontrarás un ejemplo de archivo de manifiesto:

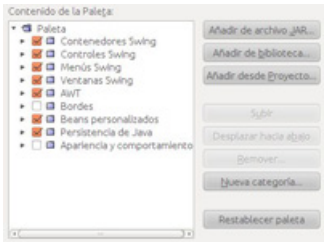
Ejemplo de archivo de manifiesto. (1.00 KB)

> En el fichero de manifiesto como la clase del componente va acompañada de `Java-Bean: True`, indicando que es un **JavaBean**.

La forma más sencilla de generar el archivo **jar** es utilizar la herramienta **Limpiar y construir** del proyecto en NetBeans que deja el fichero **.jar** en el directorio */dist* del proyecto, aunque siempre puedes recurrir a la orden **jar** y crearlo tu directamente:

```
jar cfm Componente.jar manifest.mf Componente.class ComponenteBeanInfo.class ClaseAuxiliar.class Imagen.png proyecto.jar
```

Una vez que tienes un componente Java empaquetado es muy sencillo añadirlo a la paleta de componentes gráficos de NetBeans, basta con abrir el administrador de la paleta, seleccionar la categoría donde irá el componente (normalmente en Componentes Personalizados) y seleccionar el archivo jar correspondiente.



Autoevaluación

En un archivo jar se deben incluir todos los archivos necesarios para que el componente funcione, incluidos archivos de clase, imágenes e iconos, editores de propiedades, clases `Customizer` y archivos `BeanInfo`

- ○ Verdadero.
- ○ Falso.

8.- Elaboración de un componente de ejemplo.

Caso práctico

—Por fin hemos revisado todos los aspectos relevantes de un componente, creo que ya estamos listos para empezar a teclear, —comenta **Juan**—.

Juan está encantado, ya que la programación es su aspecto favorito de la informática.

—Pues no lo creas, aunque no lo parezca, con el uso de herramientas de desarrollo gráficas prácticamente se hace en un par de clics. Se elimina mucha inserción de código a mano, —dice **Ada**, práctica como siempre—.





Para ilustrar el proceso de elaboración de un componente usando la herramienta NetBeans, vamos a crear uno muy sencillo pero completo, en el sentido de que crea un propiedad, emplea atributos internos y genera un evento.

Se trata de un temporizador gráfico que realiza una cuenta atrás con las siguientes características:

- El componente es una **etiqueta** que dispone de una propiedad llamada **tiempo** de tipo `int` que representa los segundos que van a transcurrir desde que se inicia hasta que la cuenta llega a cero.
- Cada segundo disminuye en uno el valor de tiempo, que visualizamos en el texto de la etiqueta.
- Para programarlo se utiliza un atributo de tipo `javax.swing.Timer`, que será el que marque cuando se cambia el valor de tiempo.
- Al finalizar la cuenta atrás se lanza un evento de finalización de cuenta que puede ser recogido por la aplicación en la que se incluya el componente.

Este componente se puede utilizar, por ejemplo, en la realización de test en los que el usuario tiene que contestar una serie de preguntas en cierto tiempo.

Para la **elaboración de un componente**, debes tener muy claros los **pasos** que debes dar.

1. Creación del componente.
2. Adición de propiedades.
3. Implementación de su comportamiento.
4. Gestión de los eventos.
5. Uso de componentes ya creados en NetBeans.

En las siguientes secciones verás cómo se realizan estos pasos con el ejemplo que se acaba de exponer.

8.1.- Creación del componente.

Comenzamos creando un proyecto NetBeans nuevo de tipo Java Application. Nos aseguramos de desmarcar las opciones Crear clase principal y Configurar como proyecto principal y ponemos de nombre al proyecto ProyectoTemporizador, por ejemplo.

Una vez creado el proyecto, le añadimos un archivo nuevo de tipo Componente JavaBeans (si no lo encuentras en la lista que sale en el menú contextual, haz clic en la opción Otros... para acceder al resto de tipos de archivos). Puedes llamar a la clase **TemporizadorBean** y al paquete en el que se ubicará Temporizador.



> Para que una clase se pueda considerar un componente debe implementar la interfaz `Serializable` y, además, tener un **constructor sin argumentos** que vimos eran requisitos para la creación de componentes.

El objetivo de este componente es disponer de una etiqueta con un comportamiento específico. Puesto que ya tenemos un control con parte de la funcionalidad que nos interesa, como un método para pintar el texto con el formato adecuado ya implementado, nos serviremos de el heredando nuestra clase de **JLabel**, quedando la signatura de la clase así:

```
public class TemporizadorBean extends JLabel implements Serializable
```

obviamente, tendremos que importar el paquete **javax.swing.JLabel**.

El proyecto cuenta con una propiedad de ejemplo que puedes eliminar (su declaración y los métodos **get** y **set** de la propiedad), así como un gestor de escuchadores de cambio de propiedades que no necesitaremos, quedando el siguiente código para empezar:

```
public class TemporizadorBean extends JLabel implements Serializable {
    private PropertyChangeSupport propertySupport;
    public TemporizadorBean() {
        propertySupport = new PropertyChangeSupport(this);
    }
}
```

Autoevaluación

Cuando queremos crear un componente que use características de otro, como por ejemplo una etiqueta, como en el ejemplo acudimos a...

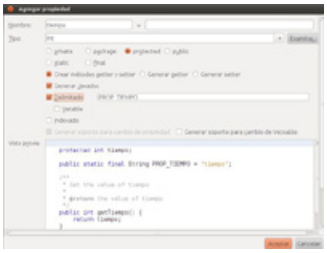
- ○ La implementación de la clase.
- ○ La herencia.
- ○ El uso de una propiedad del tipo de la clase cuyas características queremos usar.
- ○ No es posible hacer crear un componente con características de otro.

8.2.- Añadir propiedades.

El temporizador dispone de una **propiedad de lectura y escritura** en la que se almacena el número de segundos que inicialmente va a durar el temporizador y que irá disminuyendo paulatinamente cada segundo, hasta llegar a cero, momento en el que se lanzará un evento.

> La adición de propiedades en una clase Java se realiza simplemente escribiendo el código de declaración del atributo privado (o protegido) y los métodos `getter` y `setter` que son la base de la instrospección.

Si bien NetBeans proporciona una ayuda especial para realizar esta tarea. Con el cursor posicionado dentro de la clase (puedes reservar una zona para el código de propiedades) haz clic con el botón secundario y selecciona la opción **Insertar Código...**, en la lista que te saldrá elige **Agregar propiedad**. Se desplegará el siguiente cuadro de diálogo en el que puedes escribir el nombre de la propiedad y su tipo, e insertar los métodos **get** y **set** de manera automática:



> Fíjate que si quisieras crear una propiedad indexada o restringida es muy sencillo de hacer, sólo hay que marcar la opción correspondiente y la herramienta genera el código base de los métodos necesarios.

Crea la propiedad tiempo, de tipo **int**. Aunque tenemos un código añadido, será necesario modificar algo el método **setTiempo**, ya que cada vez que cambiemos el valor de tiempo será necesario reflejar ese cambio en el texto de la etiqueta y repintarla para que surta efecto, quedando su código así:

```
public void setTiempo(int tiempo) {
         this.tiempo = tiempo;
        setText(Integer.toString(tiempo));
        repaint();
    }
```

Autoevaluación

Para que una clase pueda ser considerada un componente debe implementar la interfaz`Serializable` y tener un constructor sin argumentos. ¿Verdadero o falso?

- ○ Verdadero.
- ○ Falso.

8.3.- Implementar el comportamiento.



Con la propiedad lista, tenemos que programar el comportamiento del componente. Parte de un valor inicial que irá disminuyendo cada segundo hasta llegar a cero, para implementar esto usaremos un **Timer**, que añadiremos como atributo privado de la clase. Un **Timer** se inicializa con un valor de intervalo entre acción y acción que se especifica en milisegundos, nosotros lo inicializamos a 1000 para que ejecute el método **actionPerformed** cada segundo. También hay que pasarle el objeto **actionPerformed** con el método a ejecutar, en nuestro caso, como necesitamos acceder a atributos de la etiqueta y a la propiedad tiempo, usaremos el de la propia clase.
Para que la clase pueda tener el método **actionPerformed** tendrá que implementar la interfaz **ActionListener**.

Añadiremos el siguiente código:

Código de la clase temporizador. (2.24 KB)

8.4.- Gestión de eventos.



Los componentes Java utilizan el modelo de delegación de eventos para gestionar la comunicación entre objetos como hemos visto. Para poder implementar la gestión de eventos necesitabas varias cosas:

- Una clase que implemente los eventos. Esta clase hereda de `java.util.EventObject`.
- Una interfaz que defina los métodos a usar cuando se genere el evento. Implementa `java.util.EventListener`. En este caso la gestión del evento se hará a través del método capturarFinCuentaAtras.
- Dos métodos, `addEventoListener` y `removeEventoListener` que permitan al componente añadir oyentes y eliminarlos. En principio se deben encargar de que pueda haber varios oyentes. En nuestro caso sólo vamos a tener un oyente, pero se suele implementar para admitir a varios.
- Implementar el método que lanza el evento, asegurándonos de todos los oyentes reciban el aviso. En este caso lo que se ha hecho es lanzar el método que se creó en la interfaz que describe al oyente:

Finalmente el código de la clase componente queda como aparece en el siguiente enlace:

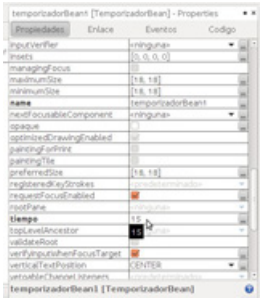
Código de la clase temporizador finalizada. (2.81 KB)

Autoevaluación

¿Qué elementos interviene en la implementación de un evento?

- ○ Una clase que define el evento.
- ○ Una interfaz que define los métodos a implementar.
- ○ Una clase que defina el evento y una interfaz que defina los métodos a implementar.

8.5.- Uso de componentes previamente elaborados en NetBeans.



Una vez construido el componente es sencillo incorporarlo a la paleta de NetBeans. Podemos hacerlo de diferentes formas:

- Si lo hemos desarrollado nosotros mismos (con NetBeans) basta con utilizar Limpiar y Construir para generar el fichero `.jar` con el componente. Cuando esté generado desde el inspector de Proyectos basta con buscarlo y con el botón secundario seleccionar *Herramientas >> Añadir a la Paleta*. Se indica en que sección debe aparecer y ya lo tenemos.
- Si es un componente generado por otras personas, es preciso disponer de la distribución en un fichero `.jar`. E incorporarlo a la paleta desde el *Administrador de la paleta* (se accede con el botón secundario sobre la paleta). También en este caso se indica la sección en la que debe aparecer.

Una vez que hayas hecho esto tendrás el componente en la paleta y lo podrás añadir a un proyecto nuevo como cualquier otro. Esto incluye el tratamiento de las propiedades, desde el inspector de propiedades tendrás acceso a las propiedades propias de una etiqueta, y también a la propiedad tiempo definida por ti. Si la modificas verás como cambia el texto de la etiqueta.

Prueba el uso del temporizador en un proyecto nuevo. La ventaja de usar una herramienta como NetBeans es que, gracias al proceso de introspección, es capaz de detectar las propiedades, métodos y eventos del componente, por lo que es muy fácil hacer la gestión del evento de finalización de la cuenta atrás, no tienes más que hacer clic con el botón secundario sobre el componente y seleccionar *eventos >> FinCuentaAtras >> CapturarFinCuentaAtras*.

Se creará automáticamente una función que implementará la gestión del evento, puedes incluir este código:

```
private void temporizadorBean1CapturarFinCuentaAtras ( Temporizador.TemporizadorBean.FinCuentaAtrasEvent evt) {
        // TODO add your handling code here:
        JOptionPane.showMessageDialog(null, "La cuenta atrás ha terminado", "Aviso",
        JOptionPane.INFORMATION_MESSAGE);
    }
```

que simplemente muestra un mensaje, pero puedes gestionar cualquier otro modo de finalizar la cuenta atrás.

Si ejecutas el proyecto verás cómo al finalizar la cuenta atrás salta el mensaje de aviso.

Para saber más

A continuación tienes el código de la clase con el temporizador y el proyecto en el que se usa.

Para que puedas estudiar y probar el código tendrás que descomprimir la carpeta en alguna carpeta de tu disco duro. Puedes hacerlo en el directorio donde se almacenan los proyectos de NetBeans. Abre los proyectos desde **Archivo >> Abrir Proyecto**. Dado que se presentan completos puedes ver el código, compilarlos y ejecutarlos sin problemas, no obstante recuerda que para usar el componente **Temporizador** en otra aplicación debes añadirlo a la **Paleta**.

Si abres el proyecto **PruebaTemp** verás el temporizador en JFrame. En el panel de **Propiedades** aparece la propiedad **Tiempo**, entre el resto que pertenecen a la etiqueta de la que procede.

Analiza también el código del **ProyectoTemporizador** y observa la función que cambia cada segundo el valor de la propiedad tiempo y que lanza el evento **FinCuentaAtras** cuando llega a cero.

Código de ejemplo. (2.40 MB)

Anexo.- Licencias de recursos.

Licencias de recursos utilizados en la Unidad de T

Recurso (1)	Datos del recurso (1)	Recurso (2)
	Autoría: Everaldo Coelho. Licencia: LGPL. Procedencia: http://commons.wikimedia.org/wiki/File:Crystal_Clear_mimetype_source_java.png?uselang=es	
	Autoría: warszawianka. Licencia: Dominio público. Procedencia: http://www.openclipart.org/detail/32407/tango-edit-find-by-warszawianka	
	Autoría: warszawianka. Licencia: Dominio público. Procedencia: http://www.openclipart.org/detail/32245/tango-documnet-save-by-warszawianka	
	Autoría: Silveira Neto. Licencia: CC by-sa. Procedencia: http://www.flickr.com/photos/silveiraneto/2579658422/	
	Autoría: Netbeans. Licencia: CDDL, GNU General Public License 2. Procedencia: Captura de pantalla de Netbeans.	